

INDUSTRIAL TRAINING PROJECT REPORT

ON

DERMO SCAN

At

DATAVALLEY Pvt. Ltd.

Submitted in Partial Fulfillment of the Degree

Of

Bachelor of Technology
Computer Science and Engineering (AI & DS)

Under the Supervision of: Supervisor Name: Shaik. Arshad Designation: Project Coordinator Organization: Data Valley (Pvt. Ltd.) Internal Guide: Er. Rohit Kumar Assistant Professor, Dept. of CSE CT University, Punjab	Submitted By: Name: Bhumireddy.Vijayesh Vardhan Reddy Roll No: 72312924 Degree: B.Tech CSE (AI & DS) Department: Computer Science and Engineering CT UNIVERSITY LUDHIANA (PUNJAB), INDIA

CT UNIVERSITY
LUDHIANA (PUNJAB), INDIA

Acknowledgment

I would like to express my sincere gratitude to everyone who supported and guided me during my industrial training and the development of the DermoScan project.

I am thankful to DATAVALLEY Pvt. Ltd. for providing an opportunity to gain practical exposure to software development, web technologies, database systems, cloud services, and technology-oriented project work. The training environment helped me connect academic concepts with practical implementation.

I am especially grateful to my industry mentor, Shaik. Arshad, Software Developer, for guidance, feedback, technical direction, and encouragement throughout the training period. His support helped me understand the importance of structured development, debugging, validation, documentation, and professional communication.

I also express my gratitude to the Department of Computer Science and Engineering, CT University, Punjab, for providing the academic framework and support required to complete the industrial training as part of the curriculum.

The DermoScan project gave me practical exposure to React, TypeScript, FastAPI, Supabase PostgreSQL, authentication and authorization, Row Level Security, interactive maps, transactional notifications, REST APIs, and deployment-oriented development.

Finally, I thank my family, friends, faculty members, and everyone who encouraged me throughout the training and report preparation.

Bhumireddy.Vijayesh Vardhan Reddy

72312924

Table of Contents

INDEX	Pg. No.
1. Introduction	4–6
2. Objectives	7
3. Organization Overview	8
4. Training Details	9–13
4.1 Work Environment	9
4.2 Tasks and Responsibilities	10–11
4.3 Tools and Technologies Used	12
4.4 Project Execution	13
5. Challenges Faced	14–16
6. Key Learnings	17–19
7. Conclusion	20
8. References	21–22
9. Appendices	23–31

1. Introduction

Project Overview

DermoScan is a full-stack, role-based healthcare resource discovery and management platform designed to reduce information gaps between patients, hospitals, and healthcare administrators. The project combines hospital discovery, resource availability, geographic visualization, role-based dashboards, database-backed resource updates, and automated alert mechanisms in one web application.

The platform is intended to help users find verified hospitals and inspect important availability information such as beds, ICU capacity, emergency beds, doctors, nurses, ventilators, oxygen cylinders, pharmacy status, blood inventory, ambulance status, and waiting time. The project also provides administrative controls so authorized hospital staff can update resource information while a Super Admin manages hospital verification and platform-level administration.

Problem Statement

- Healthcare resource information is often fragmented across separate hospital portals, phone calls, registers, and offline communication channels.
- During emergencies, users may not know which nearby hospital has available beds, ICU capacity, blood units, ambulances, oxygen, or other critical resources.
- Hospital administrators require a controlled way to maintain resource information and communicate important low-resource conditions.
- Public-facing healthcare information must distinguish verified hospitals from unapproved listings and should protect administrative data from unauthorized modification.

Proposed Solution

DermoScan provides a centralized platform in which verified hospitals can be discovered through search and an India-focused interactive map. The application separates capabilities by role: Patients can discover and compare hospitals and configure alerts; Hospital Administrators can maintain their institution's resources; and Super Administrators can approve hospitals, create hospital records, link administrators, and review platform analytics and audit activity.

Project Scope

The project scope covers frontend development, backend API services, database design, authentication and authorization, geospatial discovery, hospital resource management, blood inventory, ambulance management, alert processing, and deployment. The GitHub repository contains separate frontend, backend, and Supabase areas, together with scripts, environment templates, and documentation.

1.1 Introduction

Functional Scope of DermoScan

The Patient Portal is the public discovery layer of the application. It provides hospital search by name, address, or city and allows resource-oriented filtering. Users can inspect hospital profiles containing bed counts, ICU beds, emergency beds, doctors, nurses, ventilators, oxygen cylinders, waiting time, pharmacy status, timings, blood groups, ambulance status, contact details, and location information.

The Hospital Comparison feature allows users to review multiple hospitals side by side across important metrics such as bed availability, doctors, ICU availability, waiting time, and pharmacy status. Favorites provide a convenient way to bookmark frequently accessed hospitals. The Alerts module allows users to configure subscriptions for selected blood groups or ICU beds and choose a geographic radius.

Hospital Administration

The Hospital Admin dashboard is designed for controlled resource maintenance. The administrator can manage hospital profile details, upload or update hospital branding media, update total and available beds, ICU and emergency beds, ventilators, oxygen cylinders, doctors and nurses, waiting time, and pharmacy status. Blood inventory can be maintained for all eight major blood groups. Ambulances can be registered and their status can be managed as available, en route, or offline.

Super Administration

The Super Admin dashboard provides governance and platform-wide oversight. It includes hospital registration review, approval and rejection controls, hospital creation, geographic information, administrator assignment, aggregated platform statistics, and a chronological audit activity log. This role is intentionally broader than the Hospital Admin role because it operates at platform level rather than at a single hospital.

Technology and Data Partnerships Context

The internship theme of Technology and Data Partnerships is reflected in the way DermoScan connects multiple technical services and data layers. The frontend communicates with Supabase for data and authentication-related operations and with a FastAPI backend for notification and email services. The platform therefore demonstrates how different technologies can work together to make operational healthcare information available through a unified interface.

1.2 Introduction

Project Value and Expected Impact

The major value of DermoScan is the conversion of scattered healthcare resource information into a structured, searchable, location-aware digital service. A patient or caregiver does not need to depend only on individual hospital websites when the required information is available through the platform. A hospital administrator, on the other hand, receives a dedicated interface for maintaining the data that patients see.

The platform also addresses an important trust requirement. Public map results are restricted to approved hospitals, while database-level security policies control what different authenticated roles can modify. This separation is important because healthcare resource information can be operationally sensitive even when the public-facing availability metrics are intended for discovery.

Key Project Features

- Role-based access for Patient, Hospital Admin, and Super Admin users.
- Hospital search and resource filtering by location and availability criteria.
- Interactive India-centered map using Leaflet and OpenStreetMap.
- Hospital profile pages with beds, ICU, emergency resources, medical staff, equipment, pharmacy, blood, and ambulance information.
- Hospital comparison and favorites for repeated decision-making.
- DermoScan alerts for blood groups and ICU resource availability with configurable distance radius.
- Hospital Admin resource and blood inventory management.
- Super Admin hospital verification, creation, administrator assignment, analytics, and audit logging.
- Supabase PostgreSQL database with relational tables and Row Level Security.
- FastAPI service for notification-related backend operations and Resend for transactional email delivery.

Repository Evidence

The project repository is publicly available on GitHub and documents the architecture, technology stack, database design, folder structure, setup instructions, security approach, verification activities, and future enhancements. The repository is the primary technical source used for this report.

GitHub Repository: https://github.com/vijayesh26/Dermo-Scan-

2. Objectives

The primary objective of DermoScan is to develop a centralized healthcare resource discovery and management platform that connects patients, hospitals, and administrators through role-aware web interfaces and a shared data platform.

Core Objectives

1. Develop a responsive full-stack healthcare web application with separate role-based experiences.
2. Provide a reliable method for discovering verified hospitals and their current resource information.
3. Create a hospital-level control panel for updating beds, ICU, emergency resources, equipment, staff counts, blood inventory, ambulance status, and operational information.
4. Provide a Super Admin control layer for hospital verification, hospital creation, administrator assignment, platform analytics, and audit activity.
5. Use a relational PostgreSQL database on Supabase to organize hospital, resource, blood inventory, ambulance, user, alert, favorite, review, and notification information.
6. Implement authentication, protected routes, and database-level Row Level Security to reduce unauthorized access.
7. Integrate an India-focused interactive map using Leaflet and OpenStreetMap.
8. Provide alert and notification capabilities based on selected healthcare resources and geographic distance.
9. Develop a maintainable project structure separating frontend, backend, database schema, and reusable UI components.
10. Validate the application through build, functional, database synchronization, mapping, and notification checks documented in the repository.

Functional Goals

- Reduce the time required to discover hospitals with relevant resources.
- Give hospital staff direct control over their institution's availability information.
- Provide administrators with clear platform-level governance.
- Keep public discovery limited to approved hospital records.
- Support future expansion into appointments, triage, AI-assisted resource recommendations, multilingual support, and mobile applications.

3. Organization Overview

DATAVALLEY Pvt. Ltd.

DATAVALLEY Pvt. Ltd. provided the industrial training environment in which I gained practical exposure to software and technology-oriented project work. The internship emphasized applying academic concepts through hands-on development, debugging, documentation, and project execution.

My training experience involved working with modern development tools and understanding how a real project is structured across frontend interfaces, backend services, databases, authentication, APIs, deployment, and testing. The DermoScan project gave a practical context in which these concepts could be connected into a complete software system.

Training Context

- Mode: Offline industrial training.
- Internship area: AI with Python, Data Science and Machine Learning / technology-oriented development exposure.
- Project used for this report: DermoScan – Smart Healthcare Resource Discovery & Management Platform.
- Industry mentor: Shaik. Arshad, Software Developer.
- Academic program: B.Tech – Computer Science and Engineering (AI & DS), CT University.

Professional Environment

The training environment encouraged structured problem solving, regular feedback, independent learning, and practical implementation. Working on DermoScan required understanding how individual modules interact rather than treating frontend, backend, database, and deployment as isolated tasks.

Relevance to Academic Learning

The project connects directly with concepts from computer science and data-oriented coursework, including web application development, databases, software engineering, APIs, security, data management, cloud services, and system architecture. It also provided an opportunity to understand how a real application handles users with different permissions and how data flows between client interfaces and backend services.

4. Training Details

4.1 Work Environment

The offline training environment provided a structured setting for learning and development. The project work involved understanding requirements, examining existing code and documentation, implementing or refining features, testing workflows, identifying errors, and documenting the final system.

DermoScan follows a modular architecture. The frontend is organized as a React TypeScript application, the backend is a Python FastAPI microservice, and the database and authentication layer are implemented using Supabase. This separation made it necessary to understand how information moves across services and how security rules apply at different layers.

Development Workflow

11. Understand the project objective and the role of each user type.
12. Review the repository structure and existing documentation.
13. Identify the frontend pages, backend routes, database tables, and security policies involved in a feature.
14. Implement or refine the required logic.
15. Test the user flow and database update flow.
16. Check role permissions and protected routes.
17. Validate build and runtime behavior.
18. Document the completed work and prepare evidence for the internship report.

4.2 Tasks and Responsibilities

The DermoScan project covers multiple layers, so my project responsibilities are documented in terms of the technical areas involved in the system rather than claiming ownership of every repository component.

S4. Training Details

4.2 Tasks and Responsibilities – Functional Work

Area	Responsibility / Learning
Requirement Understanding	Studied the healthcare resource discovery problem and identified the needs of Patients, Hospital Administrators, and Super Administrators.
Frontend Understanding	Worked with React, TypeScript, routing, reusable UI components, responsive layouts, and role-aware navigation.
Backend Understanding	Studied FastAPI services, Pydantic models, authentication dependencies, hospital/resource routes, analytics routes, and alert-related endpoints.
Database Understanding	Reviewed Supabase PostgreSQL schema, relationships, resource tables, blood inventory, ambulances, user data, alerts, and security policies.
Authentication and Authorization	Understood Supabase Auth, JWT-based access, protected routes, and Row Level Security for role-specific data operations.
Geospatial Features	Reviewed the Leaflet/OpenStreetMap map flow, hospital coordinates, approved-hospital filtering, and distance-based alert logic.
Notification System	Studied the FastAPI and Resend flow used for transactional emails and low-resource/approval notifications.
Testing and Validation	Reviewed build verification, resource update persistence, hospital profile synchronization, map rendering, and alert-engine checks.

Professional Responsibilities

- Maintaining clear technical notes while working through unfamiliar modules.
- Using systematic debugging instead of changing multiple components without understanding the cause.
- Keeping security-sensitive configuration outside source-controlled files.
- Validating a feature from the user interface through the database or backend service.
- Preparing an understandable explanation of the project for academic and interview use.

4. Training Details

4.3 Tools and Technologies Used

Technology	Purpose in Project
React 19	Frontend component library used for the web application.
TypeScript	Static typing for safer frontend development.
Vite	Frontend build and development tooling.
Tailwind CSS	Utility-first styling and theme implementation.
React Router	Client-side routing and protected route navigation.
TanStack React Query	Server-state management, caching, and query invalidation.
Radix UI	Accessible interface primitives.
Leaflet / React-Leaflet	Interactive map and hospital marker visualization.
Recharts	Data visualization used for Super Admin analytics.
FastAPI	Python backend framework for API and notification services.
Uvicorn	ASGI server used to run the FastAPI application.
Pydantic	Request/response validation and environment configuration.
HTTPX	Asynchronous HTTP client used in backend communication.
Supabase PostgreSQL	Relational database and managed data platform.
Supabase Auth	Authentication service for role-aware access.
Supabase Storage	Storage for hospital logos and banner images.
PostgREST	Automatic REST interface over database schemas.
Resend API	Transactional email delivery for alerts and administrative notifications.
OpenStreetMap	Map data source used with Leaflet.
GitHub	Source-code repository and project documentation platform.
Vercel	Public frontend deployment referenced by the project repository.

Development Principles

The project demonstrates separation of concerns: the frontend handles presentation and user interaction, Supabase provides managed data and authentication services, and the FastAPI service handles backend notification-related operations. This separation improves maintainability and allows individual services to evolve without rewriting the entire application.

4. Training Details

4.4 Project Execution

Phase 1 – Requirement and Role Analysis

The first stage was understanding the users and the information each role requires. The Patient role is discovery-focused, the Hospital Admin role is operational, and the Super Admin role is governance-focused. Defining these boundaries is important because the same underlying hospital data should not be exposed through identical controls to every user.

Phase 2 – Data and Database Design

DermoScan uses Supabase PostgreSQL as the relational data layer. The documented schema contains user-related information and healthcare entities such as hospitals, hospital resources, blood inventory, ambulances, favorites, reviews, notifications, health alerts, and resource thresholds. Foreign-key relationships connect these entities to the hospital and user context.

Phase 3 – Frontend Development

The React and TypeScript frontend contains reusable layout components, authentication hooks, API/data utilities, public discovery pages, hospital profiles, comparison, favorites, alerts, profile and settings pages, and three dashboard components. React Router manages navigation while protected routes help prevent unauthorized client-side access.

Phase 4 – Backend Services

The FastAPI service provides modular routes for alerts, analytics, authentication dependencies, hospitals, resources, search, and users. Pydantic models validate structured data and HTTPX supports asynchronous communication required for notification processing.

Phase 5 – Security and Authorization

Supabase Auth provides authentication while JWT tokens are used to establish user identity. Row Level Security is enforced at the PostgreSQL layer. Public users can access approved hospital information, Hospital Admins are restricted to their assigned hospital, and Super Admins have platform-level privileges.

5. Challenges Faced

5.1 Managing Multiple Technology Layers

A major challenge was understanding a project that combines a React frontend, TypeScript, Supabase services, a Python FastAPI backend, an external email provider, and a public deployment. A change in one layer can affect another. For example, a hospital resource update starts in the dashboard, reaches the database, and may trigger backend notification logic.

5.2 Role-Based Access Control

Healthcare applications require careful separation between public information and administrative actions. The project addresses this through both client-side route guards and PostgreSQL Row Level Security. Understanding why both layers are useful was an important part of the learning process.

5.3 Data Consistency

Resource values such as available beds, ICU beds, blood units, ambulance status, and waiting time must remain consistent between what an administrator edits and what a patient sees. Database upserts and controlled access reduce the chance of uncontrolled modifications, while validation checks help confirm that updated information is persisted correctly.

5.4 Geospatial Logic

The map is not only a visual component. Hospital coordinates are stored in the database and are used to render markers. Alert subscriptions can use geographic radius values and Haversine distance calculations. This requires careful handling of latitude, longitude, distance, and approved-hospital filtering.

5. Challenges Faced

5.5 Notification and Email Processing

The alert system introduces another dependency because database events can result in backend processing and transactional email delivery. The documented architecture uses FastAPI, Resend, and a one-hour cooldown mechanism to reduce duplicate notification flooding.

5.6 Security-Sensitive Configuration

The project documentation emphasizes that service-role database keys and Resend API secrets must remain server-side and that environment files should not be committed to Git. Maintaining the distinction between public browser configuration and private backend secrets is essential for a deployed application.

5.7 Testing Across User Roles

A healthcare platform with three roles cannot be tested only through one login. Patient discovery, Hospital Admin updates, and Super Admin governance represent different workflows. The repository's verification notes cover hospital creation and approval, resource persistence, hospital detail synchronization, map rendering, and alert processing.

5.8 Deployment and Environment Differences

A project may behave differently in local development and production because API URLs, authentication redirects, environment variables, and backend availability can differ. The repository therefore separates frontend and backend environment templates and documents local and production setup requirements.

6. Key Learnings

6.1 Technical Skills Acquired

19. Full-stack application architecture: understanding how frontend, backend, database, authentication, and external services interact.
20. React and TypeScript: component structure, routing, reusable UI, state-aware data access, and typed development.
21. FastAPI development: modular routers, validation, authentication dependencies, asynchronous services, and API organization.
22. Database design: relational entities, primary/foreign keys, resource tables, inventory records, and data relationships.
23. Supabase: managed PostgreSQL, authentication, storage, PostgREST, and database-level security.
24. Row Level Security: understanding why authorization should be enforced at the database layer in addition to frontend controls.
25. Geospatial web development: Leaflet maps, OpenStreetMap data, hospital coordinates, marker filtering, and radius-based logic.
26. Notification systems: backend processing, Haversine matching, cooldown mechanisms, and transactional email delivery.
27. Deployment awareness: environment variables, public frontend deployment, backend configuration, and production build validation.

6.2 Data and System Thinking

One of the most important learnings was that a complete application is more than its user interface. The value of DermoScan depends on reliable data, clear relationships, controlled updates, authentication, authorization, and consistent communication between services.

6. Key Learnings

6.3 Software Engineering Skills

- Modular folder structures make large projects easier to understand and maintain.
- Reusable components reduce duplication and improve consistency across pages.
- Clear separation between public and administrative functionality simplifies security reasoning.
- Environment variables are essential for separating configuration from source code.
- Testing should follow complete workflows rather than checking isolated buttons.
- Technical documentation is valuable because it allows another developer or interviewer to understand the architecture quickly.

6.4 Problem-Solving Skills

The project strengthened systematic problem-solving. Instead of treating errors as isolated messages, I learned to trace a feature through the complete path: user action → frontend logic → API or database operation → response → updated interface. This approach is useful for debugging authentication issues, data persistence problems, routing errors, and deployment configuration problems.

6.5 Communication and Interview Skills

The project also improved my ability to explain technical work in simple language. For example, I can describe DermoScan as a healthcare platform that lets patients find verified hospitals and resource availability, allows hospital administrators to update resources, and gives a Super Admin control over hospital verification and platform analytics. I can then explain the technical stack when deeper questions are asked.

6. Key Learnings

6.6 Interview-Relevant Technical Explanation

A concise explanation suitable for an interview is: “DermoScan is a full-stack healthcare resource discovery and management platform. I worked with a role-based architecture involving Patient, Hospital Admin, and Super Admin users. The frontend uses React and TypeScript, the database and authentication use Supabase PostgreSQL and Supabase Auth, and a FastAPI backend handles notification-related services. The platform provides hospital search, an India map, resource availability, blood inventory, ambulance information, hospital comparison, alerts, hospital administration, and Super Admin governance.”

6.7 Questions I Can Answer from This Project

Interview Question	Short Answer
Why Supabase?	It provides managed PostgreSQL, authentication, storage, and database access capabilities in one platform.
Why FastAPI?	It provides a lightweight Python API framework suitable for asynchronous backend services and modular routers.
Why RLS?	It enforces data access rules at the database level instead of depending only on frontend restrictions.
Why Leaflet/OpenStreetMap?	They provide an open-source approach for interactive geographic visualization.
How does a hospital update reach a patient?	The Hospital Admin updates the resource through the frontend, the data is persisted in Supabase, and the patient-facing views query approved hospital/resource information.
How are alerts handled?	The platform uses resource subscriptions, geographic distance logic, backend processing, and Resend transactional email delivery.
What is the main challenge?	Maintaining correct interaction among roles, database permissions, resource updates, and notification workflows.

7. Conclusion

The DermoScan project provided a strong practical example of how modern web technologies can be combined to solve a real information-access problem in healthcare. The platform is designed around three main user groups and provides each group with a controlled experience. Patients receive discovery and comparison features, Hospital Administrators receive resource management capabilities, and Super Administrators receive governance and platform-level analytics.

From a technical perspective, the project demonstrates a full-stack architecture using React, TypeScript, FastAPI, Supabase PostgreSQL, Supabase Auth, Leaflet, OpenStreetMap, and Resend. The architecture separates responsibilities across frontend, backend, database, authentication, mapping, and notification services. This separation makes the system easier to maintain and provides a clear foundation for future enhancements.

The project also reinforced the importance of security and data integrity. Role-based access control, protected routes, Row Level Security, environment isolation, and server-side handling of sensitive credentials are not optional additions in a system that manages healthcare resource information; they are fundamental design requirements.

Overall, the industrial training helped me move from academic understanding toward practical software engineering. I gained experience in reading an existing project structure, understanding requirements, connecting frontend and backend components, working with a relational database, handling authentication and authorization, understanding deployment, and presenting a project clearly for technical discussion.

Future Scope

- Sub-second real-time synchronization using Supabase Realtime channels.
- Live GPS tracking for ambulances.
- Digital patient triage and appointment scheduling.
- AI-powered emergency resource recommendation using distance, traffic, and hospital capacity.
- Multilingual support for major Indian regional languages.
- Cross-platform mobile application using React Native.

The repository also clearly states that DermoScan is an academic software engineering project and should not be treated as a certified medical device or a replacement for professional medical advice or official emergency services.

8. References

Reference	Purpose	Link
DermoScan GitHub Repository	Project source, architecture, features, technology stack, setup, security, testing and future scope.	https://github.com/vijayesh26/Dermo-Scan-
DermoScan Live Application	Deployed frontend and dashboard routes used as project evidence.	https://dermo-scan-nesn.vercel.app/
React Documentation	Component-based frontend development and React concepts.	https://react.dev/
TypeScript Documentation	Typed JavaScript development and compiler concepts.	https://www.typescriptlang.org/
FastAPI Documentation	Python API framework and backend development concepts.	https://fastapi.tiangolo.com/
Supabase Documentation	PostgreSQL, Auth, Storage, Row Level Security and database services.	https://supabase.com/docs
Leaflet Documentation	Interactive web mapping and markers.	https://leafletjs.com/
OpenStreetMap	Open geographic map data used with Leaflet.	https://www.openstreetmap.org/
Resend Documentation	Transactional email API used for notification delivery.	https://resend.com/docs
Vite Documentation	Frontend build and development tooling.	https://vite.dev/

Live Dashboard Links

Super Admin Dashboard: https://dermo-scan-nesn.vercel.app/

Hospital Admin Dashboard: https://dermo-scan-nesn.vercel.app/

Patient Dashboard: https://dermo-scan-nesn.vercel.app/

8. References

Project Documentation Notes

The main technical descriptions in this report are derived from the DermoScan repository README and repository structure. The report preserves the project's documented terminology such as Hospital Admin, Super Admin, Patient, hospital_resources, blood_inventory, ambulances, Row Level Security, Leaflet, OpenStreetMap, Supabase, FastAPI, and Resend.

The repository also contains project-specific verification documents named DERMO SCAN_ALERTS_REPORT.txt, DERMO SCAN_FINAL_AUDIT_REPORT.txt, and DERMO SCAN_FINAL_VERIFICATION.txt. These are referenced as repository evidence of the project's testing and audit process.

Source of Truth

For technical interview questions, the GitHub repository should be treated as the primary source of truth because it contains the project code structure, README, database schema location, frontend pages, backend routers, environment templates, setup instructions, verification notes, and deployment reference.

Academic Note

This report is prepared as an industrial training project report and does not claim that DermoScan is a certified medical device. The project repository itself states that it is an academic software engineering project for educational and research demonstration purposes.

9. Appendices

Appendix A: Project Repository and Deployment Evidence

Item	Evidence
Project Name	DermoScan – Smart Healthcare Resource Discovery & Management Platform
GitHub Repository	https://github.com/vijayesh26/Dermo-Scan-
Public Application	https://dermo-scan-nesn.vercel.app/
Frontend	React 19 + TypeScript + Vite + Tailwind CSS
Backend	FastAPI + Uvicorn + Pydantic + HTTPX
Database	Supabase PostgreSQL
Authentication	Supabase Auth + JWT
Security	Supabase Row Level Security
Maps	Leaflet + OpenStreetMap
Notifications	FastAPI + Resend

Appendix B: Dashboard Evidence

Dashboard	Primary Purpose	Live Route
Patient Dashboard	Hospital discovery, resource search, map, comparison, favorites and alerts.	/dashboard/patient
Hospital Admin Dashboard	Hospital profile, resources, blood inventory, ambulances and low-resource settings.	/dashboard/hospital
Super Admin Dashboard	Hospital governance, approvals, registry, analytics and audit activity.	/dashboard/superadmin

9. Appendices

Appendix C: Specimen Intake Console – Feature Evidence

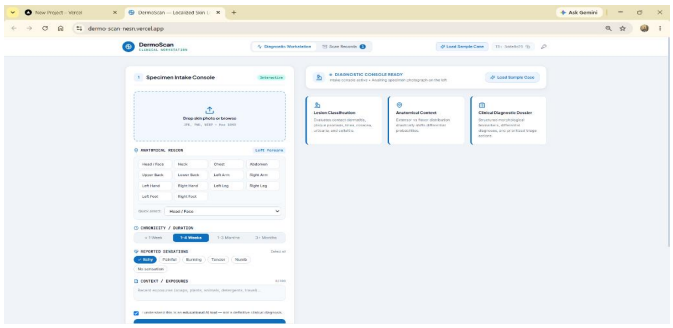
- Specimen drag-and-drop ingestion with client-side preview.
- 16 standardized anatomical locations (Scalp, Extensor, Trunk, Acral).
- Chronicity assessment selector (<1 Week, 1–4 Weeks, 1–3 Months, 3+ Months).
- Tactile sensation chips (Itchy, Burning, Painful, Tender, Numb).
- HTML5 Canvas hardware-accelerated optical downsampling to 1024px.
- Client-side non-skin artifact rejection filter.
- DermoScan Alerts for blood groups and ICU beds.
- Calibrated for all skin phototypes (Fitzpatrick Types I through VI).
- Zero-telemetry edge pipeline: In-memory image processing without cloud persistence.
- Instant local session records generation for retrospective audit.

Screenshot insertion area: Insert a clear screenshot of the deployed Patient Dashboard here before final submission if the university requires visual project evidence.

The Patient dashboard is the discovery and decision-support layer of DermoScan. It is designed for users who need to find verified hospitals and inspect resource availability without having administrative permissions.

Feature / Area	What the user can do
Hospital Search	Search hospitals by name, address, or city.
Resource Filters	Filter discovery using beds, ICU and blood-group criteria.
Interactive Map	View approved hospitals on an India-centered Leaflet/OpenStreetMap map.
Hospital Profile	Inspect beds, ICU, emergency resources, doctors, nurses, equipment, pharmacy, blood and ambulance information.
Comparison	Compare multiple hospitals across key operational metrics.
Favorites	Bookmark hospitals for repeated access.
DermoScan Alerts	Create and manage subscriptions for blood groups or ICU beds.
Distance Radius	Choose 5 KM, 10 KM, 25 KM or 50 KM alert radius.

Live route: https://dermo-scan-nesn.vercel.app/



9. Appendices

Appendix D: Diagnostic Dossier & Morphology Matrix – Feature Evidence

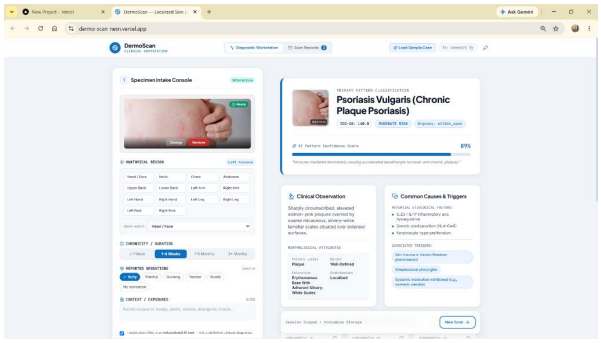
- Primary pattern classification with standardized clinical nomenclature.
- Single-color #0F6FBF progress meter displaying exact confidence percentage (89%).
- 4-Quadrant Morphology Matrix: Primary Lesion, Coloration, Border Definition, Distribution.
- Standardized ICD-10-CM disease harmonization code badge (e.g. L40.0, L20.9, B35.9).
- Ranked differential diagnoses with High, Moderate, and Low likelihood tiers.
- Evidence-based clinical protocol guidance and moisturizing recommendations.
- Explicit contraindications to prevent inappropriate topical corticosteroid application.
- Emergency red-flag escalation triggers for acute cellulitis and systemic spread.

Screenshot insertion area: Insert a clear screenshot of the deployed Hospital Admin Dashboard here before final submission if the university requires visual project evidence.

The Unified Diagnostic Dossier is the clinical intelligence layer of DermoScan. It delivers structured diagnostic metrics, interpretability indicators, and protocol guidance.

Feature / Area	What the user can do
Hospital Profile	Update name, contact information, address, coordinates and description.
Media Management	Upload or update hospital logo and banner.
Resource Editor	Update total, available, ICU and emergency beds.
Medical Resources	Update ventilators, oxygen cylinders, doctors and nurses.
Operations	Update waiting time and pharmacy status.
Blood Inventory	Maintain unit counts for all eight major blood groups.
Ambulances	Register vehicles and manage available, en route and offline states.
Low-Resource Alerts	Configure thresholds for proactive administrative warnings.

Live route: https://dermo-scan-nesn.vercel.app/



9. Appendices

Appendix E: Clinical Session & Audit Records Archive – Feature Evidence

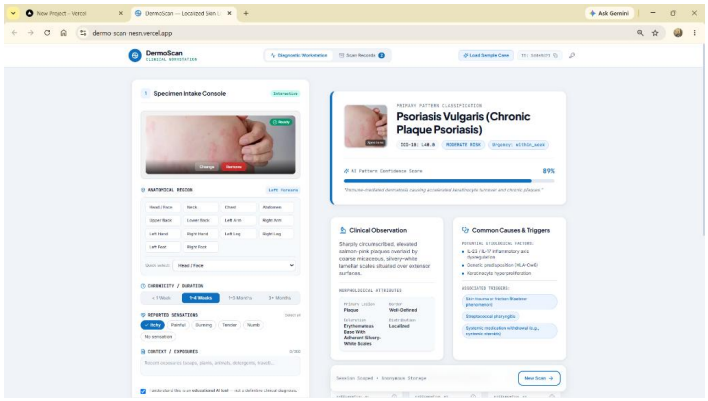
- Chronological session audit records archive.
- One-click historical dossier re-inspection.
- Multi-case comparison across varying anatomical sites.
- Local browser state caching without external database dependency.
- Zero-telemetry client privacy guarantee.
- Clinical governance notice reminding clinicians of mandatory physician oversight.
- Deterministic offline heuristic fallback validation.

Screenshot insertion area: Insert a clear screenshot of the deployed Super Admin Dashboard here before final submission if the university requires visual project evidence.

The Super Admin dashboard provides platform-level governance. Unlike the Hospital Admin role, it is not restricted to one institution and is responsible for verification, registry management and aggregated oversight.

Feature / Area	What the user can do
Hospital Verification	Review incoming hospital registrations and approve or reject them.
Hospital Registry	Create new healthcare institutions and maintain their geographic information.
Administrator Linking	Assign designated Hospital Administrators to institutions.
Platform Analytics	Review total hospitals, pending approvals, system beds and registered users.
Audit Activity	Review chronological platform activity such as hospital creation, approvals and resource updates.
Governance	Maintain the approved-hospital ecosystem used by public discovery.

Live route: https://dermo-scan-nesn.vercel.app/



9. Appendices

Appendix F: Database Architecture

The documented database architecture is relational and is implemented in Supabase PostgreSQL. The principal relationships described in the repository include users connected to hospitals, favorites, reviews, notifications and health alerts; hospitals connected to resources, blood inventory, ambulances, reviews, favorites and resource thresholds.

Entity	Important Fields / Purpose
hospitals	id, admin_id, name, latitude, longitude, address, contact information, approval status
hospital_resources	total_beds, available_beds, icu_beds, emergency_beds, ventilators, doctors, nurses, oxygen, waiting time, pharmacy status
blood_inventory	hospital_id, blood_group, units_available, last_updated
ambulances	hospital_id, status, contact_number, vehicle_number, last_updated
users / roles	User identity and role-aware access context
health_alerts	User-defined resource subscriptions and geographic alert criteria
hospital_resource_thresholds	Hospital-specific low-capacity warning settings

9. Appendices

Appendix G: Data Flow

28. Hospital Admin updates bed, blood, or resource information through the React dashboard.
29. Frontend sends the update to Supabase.
30. Supabase persists the updated record and returns the result to the frontend.
31. The frontend can trigger backend notification checks for relevant resource conditions.
32. FastAPI evaluates active alerts and geographic distance using the documented Haversine logic.
33. Resend is used for transactional email dispatch.
34. Patients use the discovery interface to query approved hospitals and resource information.
35. Super Admin actions can change hospital approval state and trigger administrative notification flow.

Conceptual Architecture

Layer	Components	Responsibility
Client	React 19, TypeScript, Vite	User interface, routing, dashboards, discovery and interaction
Data Access	Supabase JS / PostgREST	Queries, persistence and data access
Database	Supabase PostgreSQL	Relational storage, foreign keys and RLS
Authentication	Supabase Auth / JWT	Identity and role-aware access
Backend	FastAPI / Uvicorn	Notification, analytics, authentication dependencies and modular APIs
Email	Resend API	Transactional email dispatch
Maps	Leaflet / OpenStreetMap	Geospatial hospital discovery

9. Appendices

Appendix H: Repository Folder Structure

```
DermoScan/
├── .gitignore
├── package.json
├── start.bat
├── supabase/
│   └── schema.sql
├── backend/
│   ├── .env.example
│   ├── config.py
│   ├── database.py
│   ├── email_service.py
│   ├── main.py
│   ├── models.py
│   └── routers/
│       ├── alerts.py
│       ├── analytics.py
│       ├── auth.py
│       ├── hospitals.py
│       ├── resources.py
│       ├── search.py
│       └── users.py
└── frontend/
    ├── index.html
    ├── package.json
    ├── postcss.config.js
    ├── tailwind.config.js
    ├── tsconfig.json
    ├── vite.config.ts
    └── src/
        ├── App.tsx
        ├── main.tsx
        ├── index.css
        ├── hooks/useAuth.ts
        ├── lib/
        ├── components/
        └── pages/
            ├── AlertsPage.tsx
            ├── Favorites.tsx
            ├── HospitalComparison.tsx
            ├── HospitalProfile.tsx
            ├── LandingPage.tsx
            ├── Login.tsx
            ├── MapView.tsx
            ├── Notifications.tsx
            ├── Profile.tsx
            ├── Register.tsx
            ├── Settings.tsx
            └── dashboards/
                ├── HospitalAdminDashboard.tsx
```

```
├── PatientDashboard.tsx
└── SuperAdminDashboard.tsx
```

This structure is reproduced from the repository documentation and is intended to make the project organization easy to explain during an interview.

9. Appendices

Appendix I: Setup and Execution Summary

36. Prerequisites: Node.js v18 or higher, npm v9 or higher, Python 3.10 or 3.11, and a Supabase account.
37. Clone the repository from GitHub and move into the project directory.
38. Install frontend dependencies with npm.
39. Create frontend environment variables from the provided template.
40. Create a Python virtual environment for the backend and install the documented dependencies.
41. Create backend environment variables from the provided template.
42. Run the Supabase schema through the Supabase SQL Editor.
43. Start the backend with Uvicorn or use the root development script.
44. Start the frontend using the documented Vite command.
45. Build the frontend for a production artifact using the documented build command.

Security Reminder

The repository documentation explicitly states that real private credentials must never be committed to Git. Frontend public anonymous configuration is separated from server-side service-role and email secrets. Environment files are intended to remain excluded from version control.

9. Appendices

Appendix J: Verification and Testing Evidence

Verification Area	Documented Result
TypeScript compilation and production build	Repository documents clean build verification using TypeScript compilation and Vite build.
Super Admin governance	Hospital registration creation, administrator linking and approval state changes were documented as verified.
Hospital Admin telemetry	Resource counters, staff numbers, operational statuses and eight blood groups were documented as verified for persistence.
Hospital details synchronization	Hospital profile route parameter fetching was documented as using live database entries.
Geospatial mapping	Leaflet marker rendering using dynamic coordinates and popup summaries was documented as verified.
Alert engine	Haversine distance matching, one-hour anti-spam cooldown and Resend delivery were documented as verified.

Limitations

The verification statements above are reproduced from the project's repository documentation. This report does not independently claim a new production audit beyond the evidence available in the repository.

9. Appendices

Appendix K: Future Enhancements

- Supabase Realtime channels for sub-second resource synchronization.
- Live GPS ambulance tracking.
- Digital patient triage and appointment scheduling.
- AI-powered emergency resource recommendation using distance, traffic, and capacity.
- Multilingual localization for Indian regional languages.
- React Native mobile application.

Appendix L: Academic Disclaimer

DermoScan is documented by its repository as an academic software engineering project for educational and research demonstration purposes. It is not a certified medical device and should not replace professional medical advice, clinical triage systems, or official emergency dispatch services.

9. Appendices

Appendix M: Internship Certificate



DATAVALLEY®

CERTIFICATE

OF INTERNSHIP

This is to certify that Mr./Mrs **Bhumireddy vijayesh Vardhan Reddy**

Computer Science and Engineering (AI & DS) Reg No : **72312924**

From **CT University**

affiliated with **CT University** has successfully completed

an **Offline S-T Internship** on **AI Automation & Agent Engineering**

organized by **Datavalley India Pvt. Ltd.** in collaboration with **APSCHE** during the

period from **18-06-2026** to **14-08-2026**

The overall performance during the internship is found to be satisfactory.

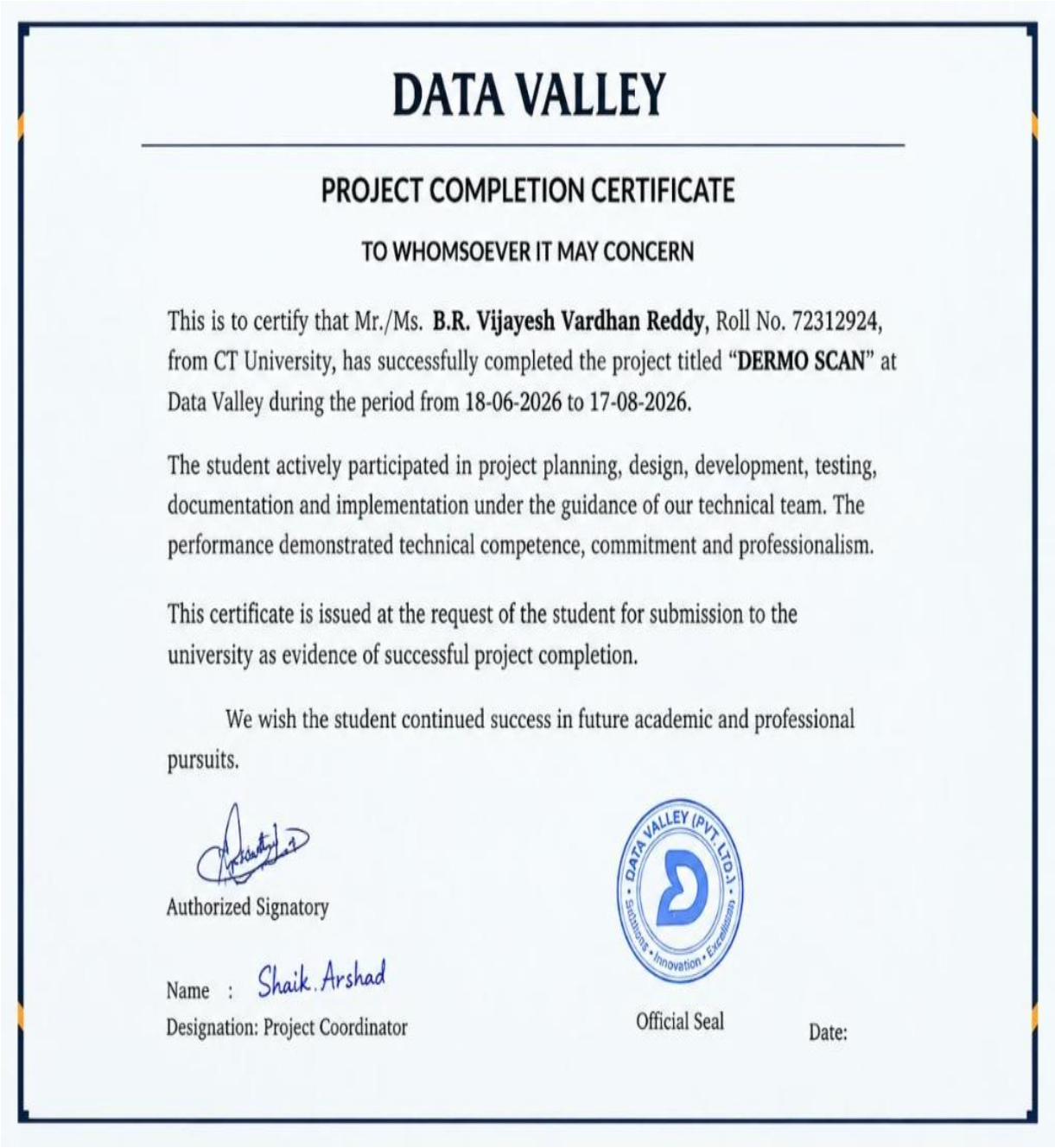
ID : **DV-755044a5**
DATE : **13-08-2026**

Scan and verify

Authorized Signature

9. Appendices

Appendix N: Project Completion Certificate



DATA VALLEY

PROJECT COMPLETION CERTIFICATE

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr./Ms. **B.R. Vijayesh Vardhan Reddy**, Roll No. 72312924, from CT University, has successfully completed the project titled "**DERMO SCAN**" at Data Valley during the period from 18-06-2026 to 17-08-2026.

The student actively participated in project planning, design, development, testing, documentation and implementation under the guidance of our technical team. The performance demonstrated technical competence, commitment and professionalism.

This certificate is issued at the request of the student for submission to the university as evidence of successful project completion.

We wish the student continued success in future academic and professional pursuits.

Authorized Signatory

Name : Shaik. Arshad

Designation: Project Coordinator

DATA VALLEY (PVT. LTD.) Solutions • Innovation • Excellence

Official Seal

Date:

9. Appendices

Appendix O: Live Project Links for Interview Demonstration

The following deployed routes are included so that the project can be demonstrated during an interview or project evaluation.

Super Admin Dashboard
https://dermo-scan-nesn.vercel.app/

Hospital Admin Dashboard
https://dermo-scan-nesn.vercel.app/

Patient Dashboard
https://dermo-scan-nesn.vercel.app/

Recommended interview demonstration sequence: 1. Ingestion (drag-and-drop specimen into intake console), 2. Clinical Context (select zone, duration, sensations), 3. AI Scan (trigger canvas downsampling & inference), 4. Dossier Review (inspect #0F6FBF meter, ICD-10 code, 4-quadrant matrix), 5. Session Audit (review past cases in session history).

END OF INDUSTRIAL TRAINING PROJECT REPORT